

Dead-Block Prediction and Dead-Block Correlating Prefetchers

A Close Reconstruction with Revisions

Deepraj Pandey
deepraj.pandey_ug20@ashoka.edu.in
Ashoka University
NCR, India

Reuel John K
reuel.johnk_ug20@ashoka.edu.in
Ashoka University
NCR, India

ABSTRACT

Prefetchers play a significant role in improving performance and reducing data access latencies. In 2001, Lai et al. introduced a prefetching technique which predicted when a cache block becomes evictable or "dead" and used that information along with instruction trace correlation to predict a prefetch address. In this paper, we re-implement the dead block correlating prefetcher in a modern out-of-order simulation system. We observe and report the performance of the two decade old design with modern workloads from SPEC2017 and based on our observations, we propose a different prefetcher based on the original design that is better suited for current applications. Our prefetcher reports a 3 - 16 cycle improvement in the reported average miss latencies over the original Dead Block Correlation Prefetcher on the benchmarks we considered.

CCS CONCEPTS

• **Hardware** → **Semiconductor memory**; • **Computer systems organization** → *Architectures*; • **General and reference** → *Validation*.

KEYWORDS

Hardware prefetcher, L1 data cache, block invalidation, differential domain, memory hierarchy, encoding

ACM Reference Format:

Deepraj Pandey and Reuel John K. 2021. Dead-Block Prediction and Dead-Block Correlating Prefetchers: A Close Reconstruction with Revisions. In *Spring '20: Advanced Computer Architecture, Ashoka University*. 6 pages.

1 INTRODUCTION

Prefetching, needless to say is a design decision that has paid off in performance over the years. For workloads with and without substantial sequential accesses cache miss rates can be reduced by upto 73% and 36% respectively[6]. According to the original authors of the Dead-Block Correlating Prefetcher (hereafter referred to as DBCP), most prefetchers rely on miss-address correlation to predict and prefetch memory addresses. Such prefetchers fall under the category of MCPs or *Miss-address Correlating Prefetchers*. Miss-address correlation in brief is a trace based approach wherein one records a prior history of cache misses and uses that to correlate

and trigger a prefetch. Instead, their implementation of a prefetcher relies on identifying when a data block can be evicted by keeping track of its accesses. By auditing the given traces one can observe that there is a considerable amount of delay between the last access of a given block and a subsequent prefetch which fetches another block in its place. The DBCP aims to eliminate this time delay and avoid wasting precious cpu cycles by prefetching a replacement block promptly as soon as the to-be-evicted block has been accessed for the last time. Therefore, while both MCPs and DBCPs rely on repetitive memory behaviour, unlike MCPs, DBCPs "primarily capitalize on repetitive *instruction sequences* - rather than memory address sequences"[4].

2 RELATED WORK

We used a reference paper written by the same authors which focuses on *Last Touch Predictors* [3] which forms the backbone of the prediction algorithm in the DBCP. We borrow multiple design decisions from their implementation of the LTP which enabled us to produce an accurate replica of the original DBCP.

3 STRUCTURE

Briefly, the DBCP uses a combination of two tables, a history table and a Dead-Block correlation table.

In their paper, they also mention that even if the block is evicted prematurely, and the processor has to wait 200 cycles to get the data, it still results in a better overall performance because dead blocks have been observed to sit in cache for hundreds of thousands of cycles before being accessed again. That is valuable space being wasted for data that is not in use and won't be used for, in L1 cache timelines, a lifetime. The hundreds of cycles that we otherwise save by getting actual useful data in place of a dead block amortises the premature evictions.

3.1 History Table

The history table is responsible for storing the traces we had mentioned earlier. It contains two main components that is the L1 tag array and a unique signature that is used to identify a given trace. Every trace contains two PCs (Program Counter), PC_i and PC_j . PC_i denotes the PC at which a given block X was requested from memory and resulted in a cache miss. On doing so, the cache block is now "live" as it is brought into the cache. PC_j on the other hand is the PC at which block X is accessed for the last time. If a given trace signature is identified with a dead block, the signature along with the "replacement block" is inserted into the Dead-Block Correlation Table.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Spring '20, May 05, 2020, Ashoka University
© 2021 Association for Computing Machinery.

3.2 Dead-Block Correlation Table

The Dead-Block correlation table also stores encoded traces, however limited to only those that end up with a dead-block. The table has a corresponding column which contains the address of the block that replaces the original block after it is evicted. Thus, this table is used as a supplement for the prefetcher to identify which block to prefetch. To ensure that the “new block” is predicted with confidence, a two bit saturating counter is used. Every time a new trace is encoded, the signature is looked up in the correlation table to identify whether the given trace ends up with a dead block. If there is a hit in the table, then the corresponding block is replaced with the “new block” that is correlated with the given trace.

4 IMPLEMENTATION

We use ChampSim[1] as our simulation environment to build the prefetchers and collect data for all the experiments. ChampSim has a modular setup that allows us to swap specific parts out to keep the same test environment for all our test environments. It has provisions for keeping the same prefetchers and LLC replacement policy and switching between a single core and a multicore processor which we use for a preliminary study into the multicore performance comparison. We wrote an implementation for the original DBCP proposal as well as the prefetcher with our modifications, and added them as separate L1D prefetcher files for the simulator. In this section we go into the details of both these prefetchers and discuss our design choices.

To keep the setup simple and have the ability to switch between the two models, we added the history table and the deadblock correlation table as private data members for class Cache. The reason for this design choice will become apparent when we discuss the implementation of the original DBCP. We have two different L1D prefetcher files, one each for both the models where we initialise the structures. The history table stores a signature encoding corresponding to a particular tag. We need a dictionary-like structure for this use case, so we use the C++ standard template library’s map container[7]. History table is thus a `std::map` of two `uint_64t` numbers, the block tag and the signature. The dead block correlation table needs to be indexed by the signature which is the value for the corresponding tag in history table. DBC table stores the prefetch address and a counter for every trace encoding that becomes “dead”, i.e. gets evicted from the cache. We use a map between the signature (`uint64_t`) and a `std::pair` comprising of a 2-bit saturating counter (`uint_8t`) and the predicted address (`uint64_t`). The history table is indexed by the tag bits and the DBC table is indexed by the signature.

4.1 Original DBCP

The original implementation of deadblock correlating prefetchers kept only signatures starting with *critical misses* in the DBC table. Non-critical misses are the L1D misses which eventually get a hit in L2C. The ones which miss in L2 and end up going to the memory are considered critical misses and the traces of instructions that start the block’s “live” state with a non-critical miss are removed from the DBC table. It is important to note that we get the information about whether a miss that brought a block into the cache was critical or not right when we get the first program counter trying to access the

address. Instead of waiting for the block to be evicted and moved into DBC table, we decided to remove it from the history table when we see a miss from L1D getting a cache hit in L2.

Since there are no direct inter-cache communication channels in ChampSim, we implemented the logic to remove non-critical misses as a prefetcher for L2C. It’s important to note that the function `l2c prefetcher_operate` in L2 prefetcher is called for *each L2 lookup operation*, both hits and misses. We use the functionality of `operate()` to handle non-critical misses. On every cache hit, the L2 prefetcher code checks if an entry for the tag of the block being accessed is present in the history table. If the tag is found in the history table, we delete the entry since this is a non-critical miss by virtue of it being a hit in L2. This way, we ensure that signatures starting with non critical misses never make it to the DBC table. As we discussed above, we make the design decision to make history table a private member of class Cache so it’s accessible to both levels.

A critical part of the prefetcher design is generating the signature. The original paper used truncated addition to store an encoding of the PC trace. The idea is to take 13 of the least significant bits and perform a bitwise addition with the same number of LSBs of the next PC in the trace. The 2001 DBCP paper does not mention the exact method of signature generation but we got confirmation from one of the original authors regarding the encoding algorithm. In the prefetcher code, the number of bits is set up as a macro because we also run a few experiments changing the number of bits to validate their claim that 12-13 provides the right balance when it comes to capturing enough entropy to avoid DBC table data corruption while still being small enough to be practical.

We will explain the working of the history and the dead block correlation table in the form of a control flow.

- When there’s a cache miss, assign a new row in the history table for the tag of the current block and store the 13 LSBs of the PC that resulted in this call.
- On every subsequent hit, we check if the tag exists in the history table and add the current PC to the existing signature.
- We then get the current signature for this tag from history table and use that to index into the DBC table.
- If the signature exists in DBC & the 2-bit saturating counter is ≥ 2 (signifying confidence), we invalidate this block and prefetch the address stored in the DBC table row corresponding to the signature.
- Every time a new block is brought into the cache, we also have the address of the block getting evicted. We pick the signature from the history table entry corresponding to the tag of this block.
- If we find a match for that signature in DBC table, we increment the saturating counter.
- If an entry for this signature does not exist, then we add it to the DBC table, initialise the counter to 1, and set the predicted address as the address of the block that is coming into the cache right now.

4.2 Updated DBCP

The updated prefetcher that we propose is completely based on the original design. We have only changed the encoding and the notion of when a block becomes “live”. In the interest of brevity, in

this subsection, we only highlight the changes we introduced to the original model.

- Instead of using truncated addition for creating the signature, we have used XOR. Every time a new PC is added to the trace, we XOR it with the previous signature. The signature is initialised to just the first PC and the second signature onwards is an XOR of the previous signature.
- Instead of adding a new entry to the history table only when there is a miss, we add a new entry if the tag does not already exist in the history table, irrespective of hit or miss status.
- We do not remove the signatures starting with non-critical misses from either DBC or history table. This in practical terms, essentially means we only have a prefetcher file for L1D and do not consider the L2C file that removes the entries on L2 hit.

Everything else remains the same - the assumptions and the structures of history and DBC tables, the predicting address, the 2-bit saturating counter that denotes the prediction confidence, the invalidation on DBC match of current signature, and block eviction logic. We have discussed at length the simulations we ran and the observations we made to decide on this particular design as our updated proposal in section 5.2.

5 METHODOLOGY

Before writing the code, we outlined what were the different data types and data structures that we would use. We also charted a rough control flow of the implementation by delineating the necessary functions to implement. Then we built the history table, set up the encoding schemes we will use for our experiments, and the dead-block table. After ensuring the structures work as expected, we added the predicted addresses to the DBC table along with a counter for every entry.

In doing so we experimented with different entry requirements for a block to be encoded in either or both of the tables.

5.1 Naming

We established a naming convention to keep track of our results. Every model followed a naming convention as follows : [block entry threshold - encoding function - critical miss requirement]. The possible values are:

1. Block entry threshold
 - a. M : Miss. Start encoding a trace only when the first reference to a block is a miss.
 - b. A : All. Start encoding a trace regardless of the first reference to the block.
2. Encoding function
 - a. TA - Truncated Addition.
 - b. XR - XOR encoding.
3. Critical miss requirement
 - a. C - Keep only the critical misses in context. We discuss this in section 5.
 - b. NC - Keep any of the misses regardless if they are critical or not.

The model [M-TA-NC] is our implementation of the original DBCP design. We then ran some simulations by changing the model to find a prefetcher that performs better than the original prefetcher.

5.2 Decisions for Updated DBCP Design

The original paper was written almost two decades ago. Everything from application expectations and memory footprint to cache sizes have changed drastically over time. Needless to say, what worked then might not work well today. In this section we do a comprehensive review to find the lacunae in the original design in context of today's application expectations. We picked 36 traces (see Appendix A.2 for list) from SPEC2017 across multiple domains and ran simulations by trying different prefetcher designs based on the permutations of options we listed in section 4.1 for block entry threshold, encoding function, and critical miss requirements. The design of our update was based on the data we collected.

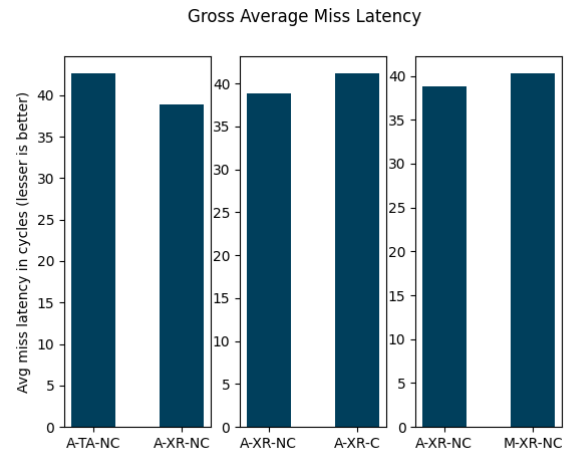


Figure 1: Mean of average miss latency values observed over 36 traces compared against multiple prefetcher designs. Lower is better.

Figure 1 highlights the gross average miss latency of all 36 traces for the different prefetcher designs we ran the simulations for. We started by fixing two arbitrary features - block entry threshold and critical miss requirement. The first decision was thus, between truncated addition and xor for encoding. Xor had a 8.82% lower L1D miss latency than truncated addition. For the next set, we fixed the encoding (xor), and the block entry threshold to compare the critical miss requirement. We wanted to see how the metrics were affected when we didn't remove the non critical misses. We observed that not removing the non-critical misses like the original design specified actually improves the performance. Keeping the non-critical misses results in a 5.61% lower average miss latency than when we remove the L2 hits. With two of our features decided fixed, we then ran simulations for [A] and [M] type block entry thresholds and found that when we start encoding a trace with only a miss like in the original specification, the miss latency is 3.53% higher than the latency observed in the design where we remove that check.

The updated model design we chose starts encoding traces whenever it sees an access to the block, uses XOR for encoding, and includes non critical misses in the history table and the dead block correlation table. From this point onwards, when we mention DBCP, we mean [M-TA-C] and when we mention the updated DBCP, we mean [A-XR-NC].

6 EXPERIMENTS AND RESULTS

We evaluated our model based on a wide variety of SPEC2017 benchmarks. These benchmarks ranged from a wide variety of workloads covering multiple memory footprints, opcode distribution and dynamic instruction counts. We finalized a list of 10 benchmarks that covered these wide range of characteristics based on the SPEC2017 workload characterization paper by Sarabjeet Singh and Manu Awasthi[5]. For all the experiments, we set up the simulations to run 100M instructions for warmup and 500M instructions for detailed simulations. A link to the directory with the collected raw data is included in appendix A.1.

In Figure 2 we see that our model performs on par, with the authors' original model. However in our case IPC by itself is not a good indicator of the prefetcher performance. While it is important, performance is essentially work divided by wall-clock time, and "number of instructions" is not a good metric for quantifying "work" except for limited use cases[2]. Thus we considered a second metric, L1D miss latency, which is represented in Figure 3. This gives a better insight to the performance of the prefetcher as shaving off cycles in miss latency cycles is a direct result of sound prefetching.

6.1 Single Core Results

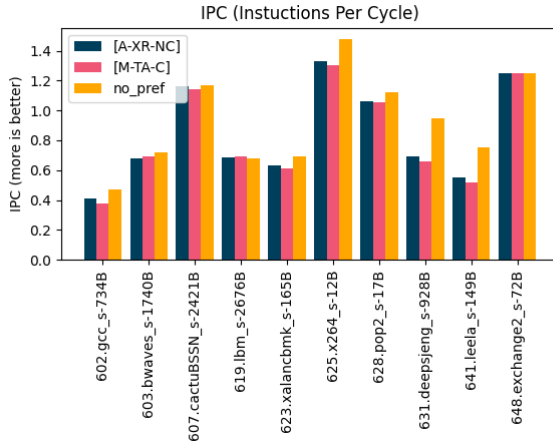


Figure 2: Instructions per Cycle (IPC) values observed for systems with the updated model [A-XR-NC], original DBCP [M-TA-C], and no prefetcher respectively for each of the 10 traces we simulated.

Figure 2 highlights the measured instructions per cycle (IPC) for systems with a) updated prefetcher [A-XR-NC], b) the original DBCP design [M-TA-C], and c) no prefetchers. We can see that our prefetcher performs better than the original DBCP by 0 – 0.3

instructions and when we take a mean of all the reported values as shown in the second column of table 1, the performance is almost the same.

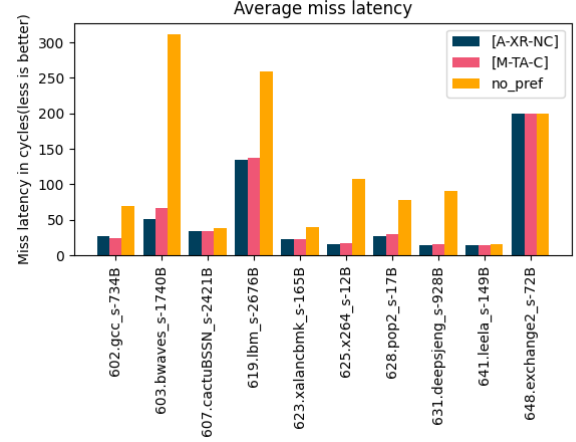


Figure 3: Average L1D miss latencies observed for systems with the updated model [A-XR-NC], original DBCP [M-TA-C], and no prefetcher respectively for each of the 10 traces we simulated. Lower is better.

However, there is an improvement when we consider the average L1D miss latency. Our model has miss latencies between 0 to 16 cycles *lesser* than the average latencies of the original model. In the third column of table 1, we have reported the mean of all values reported for all the traces. Our design has on an average 2.2 cycle lesser miss latency when compared with the miss latencies of the original DBCP. However, the average value of the metric is not helpful in getting any valid insights into the performance. For observation, it's better to observe the performance for workloads closest to the use cases under consideration.

Table 1: Mean of single core performance metrics across all traces for both the models. Original DBCP: [M-TA-C]; updated model: [A-XR-NC].

Model	IPC	Avg L1D Miss Latency
No prefetcher	0.928	121.1
Original DBCP	0.83	56.12
Updated DBCP	0.84	53.92

In both figure 2 and 3, we do not see any difference in the reported metrics for 648.exchange2_s. This is the case because this program builds non-trivial 9x9 sudoku puzzles and has a lot of array manipulations. We noticed that the system does not even issue any prefetch requests for this benchmark. That explains the exact same value for both the metrics as these metrics depend on prefetch requests coming into the prefetcher.

6.2 Multi-core results

While ensuring peak multi-core performance was out of the scope of this paper we decided to run a few 4 core simulations with our existing model.

Our first simulation was a combination of multiple workloads each from the domain of compilers, AI, file conversion and 3d modelling.

Table 2: Miss latency across multiple domains for a 4 core system

Model	Core 0	Core 1	Core 3	Core 4
Original DBCP	80.8	52.47	59.85	721.8
Updated DBCP	82.15	47.72	56.96	673.8

Our second simulation was a combination of multiple workloads covering different workloads characteristics such as significant memory operations, high memory bandwidth, high memory footprint and dynamic instruction count.

Table 3: Miss latency for varying workload characteristics for a 4 core system

Model	Core 0	Core 1	Core 3	Core 4
Original DBCP	91.35	103.45	243.03	19.66
Updated DBCP	94.19	91.18	231.91	18.86

We observed a somewhat consistently lower average L1D miss latency of our model as opposed to the original model apart from a few discrepancies and outliers.

7 CONCLUSION

We aimed to build an implementation of the 2001 DBCP design and propose some changes that works better with today's applications. Furthermore, we wanted to gauge the full potential of the predictors and the effectiveness of the entropy they encode independently of storage exactly like the initial sections of the original study. Towards that aim, we assumed tables with an unlimited number of entries for our experiments. We showed that on certain metrics, the original design still stands strong but on metrics like cache miss latencies, there is plenty of room for improvement. The aim for this paper is to pose as a decent starting point as a working proof-of-concept for future work.

8 FUTURE WORK

We could look further into the behaviour of the prefetcher and add more conditions that need to be satisfied before predicting an address. This way, we will avoid the invalidation of the blocks still in use. An extensive review of the benchmarks is required to classify the specific type of memory intensive benchmarks that work best with our prefetcher. An even more extensive study could look at other permutations of locking the features when deciding the design choices for the prefetcher design which would be a study in itself just based on the work we did in section 5, specifically 5.2.

We considered only a handful of encoding techniques and settled on xor. We could look at other possible encoding schemes that work specifically with certain types of benchmarks in a characterisation study.

ACKNOWLEDGMENTS

To Prof. Manu Awasthi for his valuable feedback regarding the experimental set up, on the direction the project should take, and for his earlier study on SPEC2017 memory characterisation that helped us make decisions regarding the workloads we used for our experiments. And to Dr. An-Chow Lai for clarifying their original method of encoding traces to generate the signatures.

REFERENCES

- [1] ChampSim Team. 2017. *ChampSim: A Simulator for Microarchitecture Study*. ChampSim. <https://github.com/ChampSim/ChampSim>
- [2] Georg Hager. 2017. Friedrich-Alexander Universität Erlangen-Nürnberg Blogs: Thoughts on High Performance Computing. <https://blogs.fau.de/hager/archives/8015>
- [3] An-Chow Lai and Babak Falsafi. 2000. Selective, Accurate, and Timely Self-Invalidation Using Last-Touch Prediction. *SIGARCH Comput. Archit. News* 28, 2 (May 2000), 139–148. <https://doi.org/10.1145/342001.339669>
- [4] An-Chow Lai, Cem Fide, and Babak Falsafi. 2001. Dead-Block Prediction & Dead-Block Correlating Prefetchers. *SIGARCH Comput. Archit. News* 29, 2 (May 2001), 144–154. <https://doi.org/10.1145/384285.379259>
- [5] Sarabjeet Singh and Manu Awasthi. 2019. Memory Centric Characterization and Analysis of SPEC CPU2017 Suite. In *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering* (Mumbai, India). Association for Computing Machinery, New York, NY, USA, 285–292. <https://doi.org/10.1145/3297663.3310311>
- [6] Vivekanand Vellanki and Ann L. Chervenak. 1999. A Cost-Benefit Scheme for High Performance Predictive Prefetching. In *Proceedings of the 1999 ACM/IEEE Conference on Supercomputing* (Portland, Oregon, USA) (SC '99). Association for Computing Machinery, New York, NY, USA, 50–es. <https://doi.org/10.1145/331532.331582>
- [7] SC22 WG21. 2011. Working Draft, Standard for Programming Language C. <http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2011/n3242.pdf>

A APPENDIX

A.1 Links

- GitHub Page With Prefetcher Code
- GitHub Page With Scripts for Data Processing
- Collected Raw Data

A.2 Workloads for Deciding Updated DBCP

SPEC2017 Traces
600.perlbench_s-210B
602.gcc_s-734B
603.bwaves_s-3699B
605.mcf_s-665B
607.cactuBSSN_s-2421B
619.lbm_s-4268B
620.omnetpp_s-874B
621.wrf_s-575B
623.xalancbmk_s-700B
625.x264_s-18B
627.cam4_s-573B
628.pop2_s-17B
631.deepsjeng_s-928B
638.imagick_s-10316B
641.leela_s-800B
644.nab_s-5853B
648.exchange2_s-1699B
649.fotonik3d_s-1176B
654.roms_s-842B
657.xz_s-3167B
605.mcf_s-484B
607.cactuBSSN_s-3477B
623.xalancbmk_s-202B
627.cam4_s-490B
654.roms_s-523B
600.perlbench_s-570B
602.gcc_s-2226B
603.bwaves_s-1080B
603.bwaves_s-2609B
603.bwaves_s-5359B
623.xalancbmk_s-165B
623.xalancbmk_s-325B
625.x264_s-12B
627.cam4_s-490B
627.cam4_s-573B
638.imagick_s-824B

A.3 Workloads for Final Comparison

SPEC2017 Traces
602.gcc_s-734B
603.bwaves_s-1740B
648.exchange2_s-72B
628.pop2_s-17B
619.lbm_s-2676B
623.xalancbmk_s-165B
625.x264_s-12B
631.deepsjeng_s-928B
641.leela_s-149B
607.cactuBSSN_s-2421B

B CONTRIBUTION

Both the authors contributed equally to the considerable amount of discussions on the exact implementation of the prefetcher, inclusive of the design decisions we had to take into account because ChampSim is structured in a certain way. The paper was vague about certain implementation details which had to be initially remedied with educated guesses which were later corroborated with the original authors.

Both the authors also contributed equally to the experimental decisions like the order of the experiments, the number of instruction cycles for various tests, and the inference metrics. While many of the design decisions were taken based on the previous paper and feasibility, some of them had to be done with a trial and error approach. For this reason we divided running the simulations and interpreting the results which were used as data for our next design decision. Thus we took a divide and conquer approach to save time and to better enable us to individually specialize in different aspects of the project while at the same time complementing each other's effort.

Deepraj wrote the code for the prefetchers and ran the simulations for the experiments.

Reuel wrote the scripts to process the data, make the graphs, and analyse the results.

Both the authors contributed equally to the report.